

```
In [68]: import pandas as pd
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader, TensorDataset
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
import matplotlib.pyplot as plt
import seaborn as sns
```

```
In [16]: # Load the dataset
file_path = "C:\\Users\\lavan\\Desktop\\Machine Learning for Cybersecurity_ Lab 6-K
try:
    testkdd = pd.read_csv(file_path)
except Exception as e:
    print(f"Error loading dataset: {e}")
    exit()
```

```
In [18]: # Inspecting the dataset
print("Dataset Columns:")
print(testkdd.columns)
```

Dataset Columns:

```
Index(['duration', 'protocol_type', 'service', 'flag', 'src_bytes',
      'dst_bytes', 'land', 'wrong_fragment', 'urgent', 'hot',
      'num_failed_logins', 'logged_in', 'num_compromised', 'root_shell',
      'su_attempted', 'num_root', 'num_file_creations', 'num_shells',
      'num_access_files', 'num_outbound_cmds', 'is_host_login',
      'is_guest_login', 'count', 'srv_count', 'serror_rate',
      'srv_serror_rate', 'rerror_rate', 'srv_rerror_rate', 'same_srv_rate',
      'diff_srv_rate', 'srv_diff_host_rate', 'dst_host_count',
      'dst_host_srv_count', 'dst_host_same_srv_rate',
      'dst_host_diff_srv_rate', 'dst_host_same_src_port_rate',
      'dst_host_srv_diff_host_rate', 'dst_host_serror_rate',
      'dst_host_srv_serror_rate', 'dst_host_rerror_rate',
      'dst_host_srv_rerror_rate', 'label'],
      dtype='object')
```

```
In [20]: # Display the first few rows of the dataset
print("First few rows of the dataset:")
print(data.head())
```

First few rows of the dataset:

	duration	protocol_type	service	flag	src_bytes	dst_bytes	land	\
0	0	icmp	ecr_i	SF	1032	0	0	
1	0	tcp	http	SF	226	468	0	
2	0	icmp	ecr_i	SF	1032	0	0	
3	0	icmp	ecr_i	SF	1032	0	0	
4	0	tcp	http	SF	254	555	0	

	wrong_fragment	urgent	hot	...	dst_host_srv_count	\
0	0	0	0	...	212	
1	0	0	0	...	255	
2	0	0	0	...	255	
3	0	0	0	...	255	
4	0	0	0	...	142	

	dst_host_same_srv_rate	dst_host_diff_srv_rate	\
0	0.83	0.01	
1	1.00	0.00	
2	1.00	0.00	
3	1.00	0.00	
4	1.00	0.00	

	dst_host_same_src_port_rate	dst_host_srv_diff_host_rate	\
0	0.83	0.00	
1	0.01	0.04	
2	1.00	0.00	
3	1.00	0.00	
4	0.14	0.04	

	dst_host_serror_rate	dst_host_srv_serror_rate	dst_host_rerror_rate	\
0	0.0	0.0	0	
1	0.0	0.0	0	
2	0.0	0.0	0	
3	0.0	0.0	0	
4	0.0	0.0	0	

	dst_host_srv_error_rate	label
0	0	not_normal
1	0	normal
2	0	not_normal
3	0	not_normal
4	0	normal

[5 rows x 42 columns]

```
In [32]: # Display dataset info
print("\nDataset info:")
print(testkdd.info())
```

Dataset info:

```
<class 'pandas.core.frame.DataFrame'>
```

RangeIndex: 1199 entries, 0 to 1198

Data columns (total 42 columns):

#	Column	Non-Null Count	Dtype
0	duration	1199 non-null	int64
1	protocol_type	1199 non-null	int32
2	service	1199 non-null	int32
3	flag	1199 non-null	int32
4	src_bytes	1199 non-null	int64
5	dst_bytes	1199 non-null	int64
6	land	1199 non-null	int64
7	wrong_fragment	1199 non-null	int64
8	urgent	1199 non-null	int64
9	hot	1199 non-null	int64
10	num_failed_logins	1199 non-null	int64
11	logged_in	1199 non-null	int64
12	num_compromised	1199 non-null	int64
13	root_shell	1199 non-null	int64
14	su_attempted	1199 non-null	int64
15	num_root	1199 non-null	int64
16	num_file_creations	1199 non-null	int64
17	num_shells	1199 non-null	int64
18	num_access_files	1199 non-null	int64
19	num_outbound_cmds	1199 non-null	int64
20	is_host_login	1199 non-null	int64
21	is_guest_login	1199 non-null	int64
22	count	1199 non-null	int64
23	srv_count	1199 non-null	int64
24	serror_rate	1199 non-null	float64
25	srv_serror_rate	1199 non-null	float64
26	rerror_rate	1199 non-null	int64
27	srv_rerror_rate	1199 non-null	int64
28	same_srv_rate	1199 non-null	float64
29	diff_srv_rate	1199 non-null	float64
30	srv_diff_host_rate	1199 non-null	float64
31	dst_host_count	1199 non-null	int64
32	dst_host_srv_count	1199 non-null	int64
33	dst_host_same_srv_rate	1199 non-null	float64
34	dst_host_diff_srv_rate	1199 non-null	float64
35	dst_host_same_src_port_rate	1199 non-null	float64
36	dst_host_srv_diff_host_rate	1199 non-null	float64
37	dst_host_serror_rate	1199 non-null	float64
38	dst_host_srv_serror_rate	1199 non-null	float64
39	dst_host_rerror_rate	1199 non-null	int64
40	dst_host_srv_rerror_rate	1199 non-null	int64
41	label	1199 non-null	int32

dtypes: float64(11), int32(4), int64(27)

memory usage: 374.8 KB

None

```
In [24]: # Encode categorical columns and the target label
categorical_columns = ['protocol_type', 'service', 'flag']
label_column = 'label'
```

```

In [30]: # Apply Label Encoding to categorical columns
label_encoders = {col: LabelEncoder() for col in categorical_columns + [label_column]}
for col, encoder in label_encoders.items():
    testkdd[col] = encoder.fit_transform(testkdd[col])

In [34]: # Split the data into features (X) and labels (y)
X = testkdd.drop(columns=[label_column])
y = testkdd[label_column]

In [38]: # Normalize the numerical features
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

In [40]: # Split the dataset into training (80%) and test (20%) sets
X_train, X_test, y_train, y_test = train_test_split(X_scaled, y, test_size=0.2, ran

In [44]: # Convert the data into PyTorch tensors
X_train_tensor = torch.tensor(X_train, dtype=torch.float32)
y_train_tensor = torch.tensor(y_train.values, dtype=torch.long)
X_test_tensor = torch.tensor(X_test, dtype=torch.float32)
y_test_tensor = torch.tensor(y_test.values, dtype=torch.long)

# Create DataLoader objects for batching
train_dataset = TensorDataset(X_train_tensor, y_train_tensor)
test_dataset = TensorDataset(X_test_tensor, y_test_tensor)

train_loader = DataLoader(train_dataset, batch_size=32, shuffle=True)
test_loader = DataLoader(test_dataset, batch_size=32, shuffle=False)

In [50]: # Define a neural network model
class Net(nn.Module):
    def __init__(self):
        super(Net, self).__init__()
        self.fc1 = nn.Linear(41, 64) # 41 features as input
        self.fc2 = nn.Linear(64, 32)
        self.fc3 = nn.Linear(32, 2) # 2 classes: normal or not_normal
        self.relu = nn.ReLU()
        self.dropout = nn.Dropout(0.5)

    def forward(self, x):
        x = self.relu(self.fc1(x))
        x = self.dropout(x)
        x = self.relu(self.fc2(x))
        x = self.fc3(x)
        return x

In [48]: # Instantiate the model, define Loss function and optimizer
model = Net()
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=0.001)

In [58]: # Training function
def train_model(model, train_loader, criterion, optimizer, epochs=20):

```

```

model.train()
for epoch in range(epochs):
    running_loss = 0.0
    for inputs, labels in train_loader:
        # Zero the gradients
        optimizer.zero_grad()

        # Forward pass
        outputs = model(inputs)
        loss = criterion(outputs, labels)

        # Backward pass and optimization
        loss.backward()
        optimizer.step()

    running_loss += loss.item()

    print(f"Epoch [{epoch+1}/{epochs}], Loss: {running_loss/len(train_loader):.

```

```

In [60]: # Training the model
train_model(model, train_loader, criterion, optimizer, epochs=20)

```

```

Epoch [1/20], Loss: 0.5777
Epoch [2/20], Loss: 0.1924
Epoch [3/20], Loss: 0.0256
Epoch [4/20], Loss: 0.0073
Epoch [5/20], Loss: 0.0065
Epoch [6/20], Loss: 0.0030
Epoch [7/20], Loss: 0.0018
Epoch [8/20], Loss: 0.0023
Epoch [9/20], Loss: 0.0015
Epoch [10/20], Loss: 0.0010
Epoch [11/20], Loss: 0.0007
Epoch [12/20], Loss: 0.0009
Epoch [13/20], Loss: 0.0005
Epoch [14/20], Loss: 0.0008
Epoch [15/20], Loss: 0.0025
Epoch [16/20], Loss: 0.0006
Epoch [17/20], Loss: 0.0028
Epoch [18/20], Loss: 0.0004
Epoch [19/20], Loss: 0.0004
Epoch [20/20], Loss: 0.0003

```

```

In [70]: # Evaluation function to calculate all metrics
def evaluate_model(model, test_loader):
    model.eval()
    y_true = []
    y_pred = []

    with torch.no_grad():
        for inputs, labels in test_loader:
            outputs = model(inputs)
            _, predicted = torch.max(outputs, 1)
            y_true.extend(labels.cpu().numpy())
            y_pred.extend(predicted.cpu().numpy())

```

```
# Calculate accuracy, precision, recall, and F1 score
accuracy = accuracy_score(y_true, y_pred)
precision = precision_score(y_true, y_pred, average='weighted')
recall = recall_score(y_true, y_pred, average='weighted')
f1 = f1_score(y_true, y_pred, average='weighted')

print(f"Accuracy: {accuracy:.4f}")
print(f"Precision: {precision:.4f}")
print(f"Recall: {recall:.4f}")
print(f"F1-Score: {f1:.4f}")

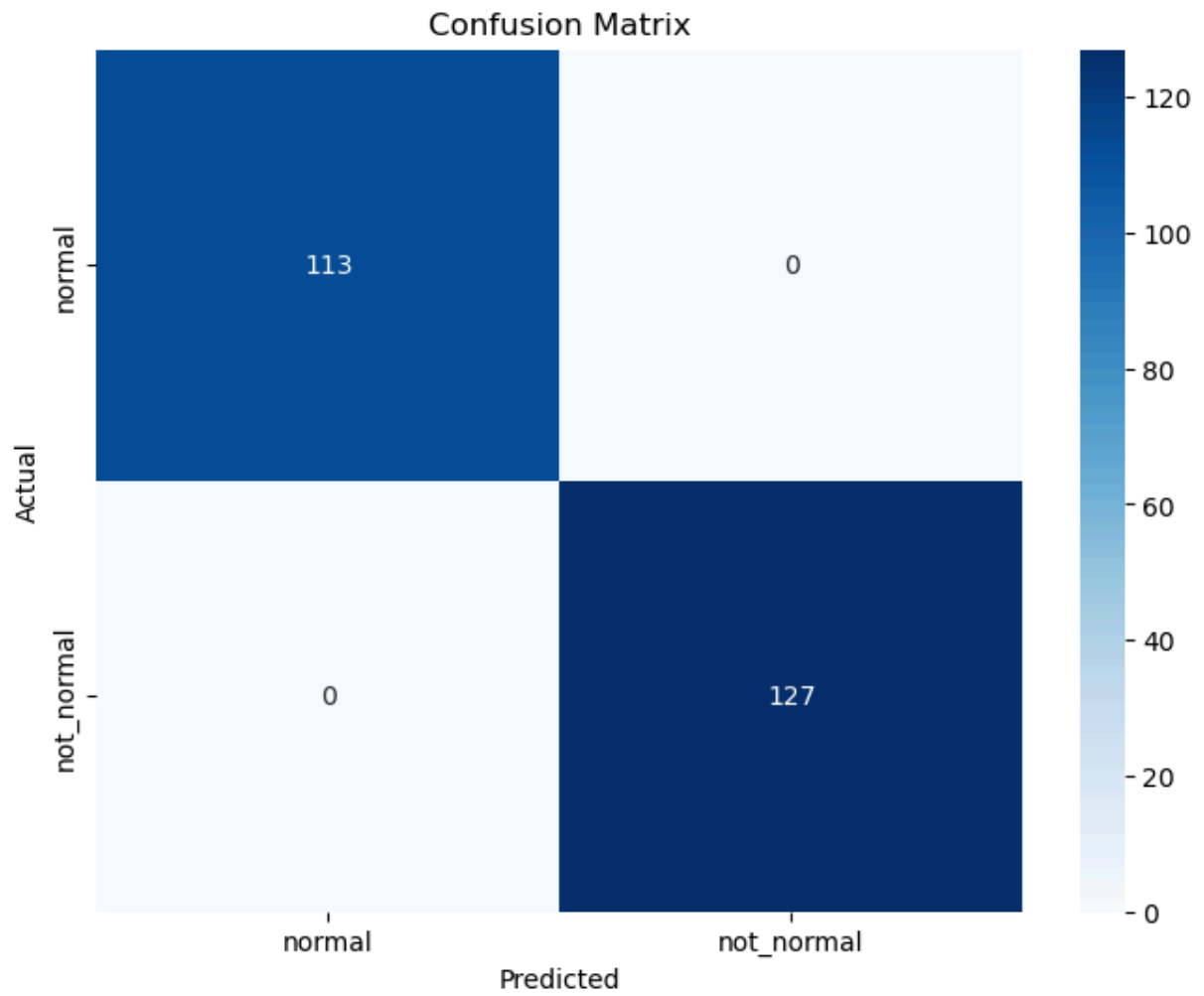
# Generate confusion matrix
cm = confusion_matrix(y_true, y_pred)
print("\nConfusion Matrix:")
print(cm)

# Plot confusion matrix
plt.figure(figsize=(8,6))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', xticklabels=label_encoders[1]
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title('Confusion Matrix')
plt.show()

# Evaluate the model on test data
evaluate_model(model, test_loader)
```

Accuracy: 1.0000
Precision: 1.0000
Recall: 1.0000
F1-Score: 1.0000

Confusion Matrix:
[[113 0]
 [0 127]]



In []: